

Universidade Federal de Viçosa – Campus Florestal
Bacharelado em Ciência da Computação

SISTEMAS DISTRIBUÍDOS

PROFESSOR(A): Thais Regina de M. B. Silva

SISTEMAS DISTRIBUÍDOS DEPLOY WITH SOCKETS:

Gabriel Santos Ferreira de Pádua - 4705



Introdução - Garden in Overworld.....	3
Organização:.....	4
Cliente:.....	5
Servidor:.....	5
Componentes (Servidor):.....	6
Camada Controllers:.....	6
ServidorTCP (ClassServidorTCP.py).....	6
ControladorFazenda (ClassControlador.py).....	6
Camada Models.....	7
Estado do Jogo/Mapa (ClassMapa.py).....	7
GerenciadorUsers e User (ClassUser.py).....	8
Camada Views:.....	9
ClassMapViewTerminal.py.....	9
mainServer.py.....	9
Componentes (Cliente).....	10
Camada Controller:.....	10
ClassControladorCliente.py e ClassOuvinte.py.....	10
Camada Models:.....	10
TCPCliente (ClassTCPCliente.py):.....	10
ClienteFazenda (ClassFazendeiro.py):.....	10
Camada View:.....	11
Interface Gráfica Pura (ClassGUI.py).....	11
mainClient.py.....	11
Troca de Mensagens.....	11
Tabela de Mensagens (Servidor para Cliente).....	12
Tabela de Mensagens (Cliente para Servidor).....	16
Uso das Threads.....	17
Threads no Servidor:.....	17
Threads no Cliente:.....	18
A Thread Oculta de Escuta (OuvinteThread).....	18
Mecanismo de Sincronização Segura via Sinais (pyqtSignal).....	18
Dificuldades:.....	20
Extras.....	20
Conclusão.....	21
Github:.....	21

Introdução - Garden in Overworld

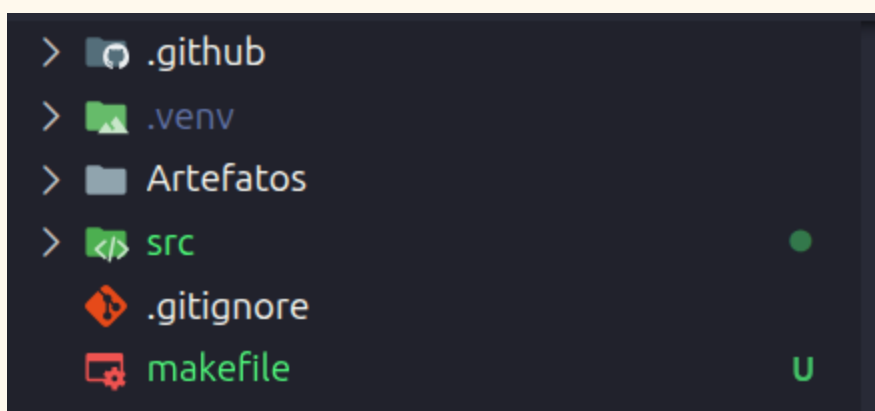
O sistema distribuído é baseado em um Mundo de Construção Livre, inspirado na estética e mecânicas do jogo *Minecraft*. O objetivo central é permitir que múltiplos usuários interajam de forma simultânea e colaborativa em um ambiente compartilhado, simulando a gestão de uma horta comunitária em um mapa bidimensional.

O sistema utiliza uma matriz de coordenadas 2D como estrutura de dados central no servidor, que inicializa o mundo com estados padrão representando diferentes biomas: terra, areia e água. A partir dessa base, os clientes conectados podem realizar operações de modificação do estado do mundo, como o plantio de culturas específicas (arroz, trigo ou cana-de-açúcar) e a remoção de estruturas existentes (colheita).

O nome Garden in Overworld vem do fato de se tratar de plantações do universo de *Minecraft*, onde o planeta/mundo é apelidado de Overworld.

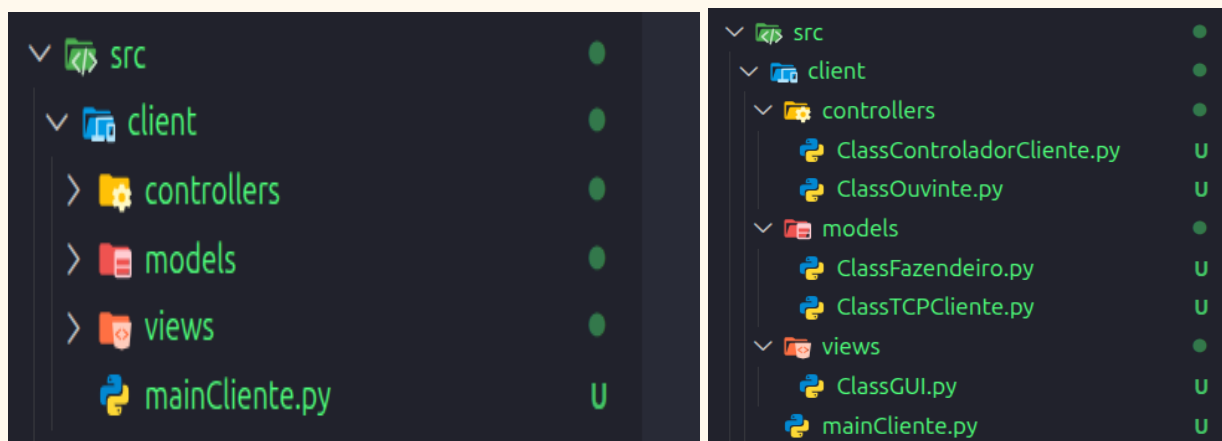
Organização:

Tanto o servidor, quanto o cliente foram projetados seguindo o padrão arquitetural de separação de responsabilidades (MVC/Injeção de Dependência). Visando isolar completamente a lógica de baixo nível da rede (Sockets e gerenciamento de buffers) das regras de negócio do jogo (validação de plantio, gerenciamento de mapa e inventários) e maneiras de visualização (frontend e terminal). Assim seguindo os padrões de qualidade de software e ciclo de vida aprendidos em Engenharia de Software.

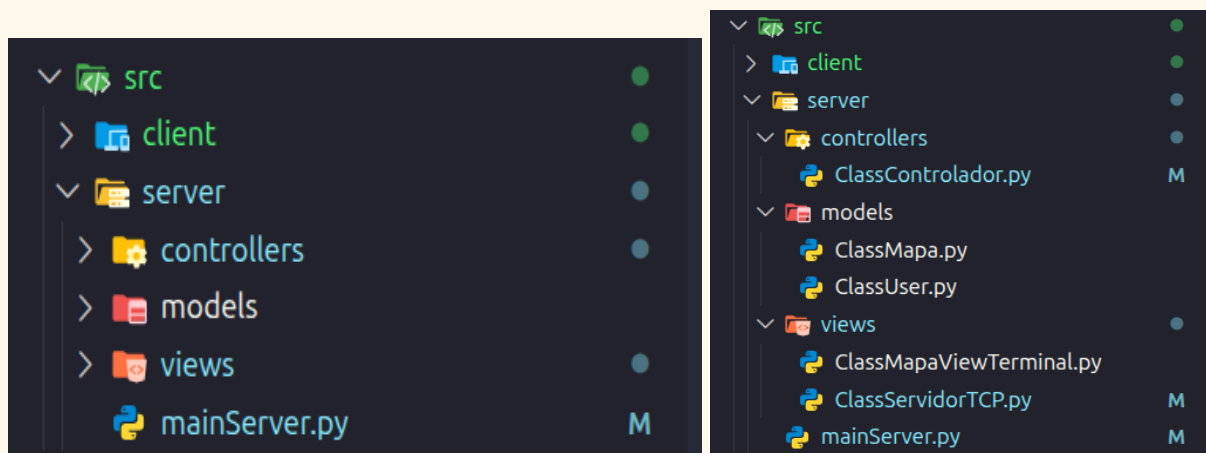


No repositório src/ encontram-se arquivos fonte para o servidor e a replicação de clientes.

Cliente:



Servidor:



Componentes (Servidor):

Camada Controllers:

ServidorTCP (ClassServidorTCP.py)

Responsabilidade: Escutar conexões de entrada, alocar threads dedicadas para cada cliente, ler dados brutos da rede e garantir o envio confiável de pacotes unidirecionais ou em formato de *broadcast* (espalhamento).

Funcionalidades e Permissões: Possui permissão para abrir a porta mestre do sistema (bind) e gerenciar o ciclo de vida físico dos sockets. Mantém um dicionário ativo de conexões (conexoes_ativas) mapeando o ID de cada jogador ao seu respectivo objeto de conexão.

Escolha(s) de Implementação Crítica (Gerenciamento de Buffer): Redes TCP fragmentam ou agrupam dados por serem orientadas a fluxo de bytes (stream). Para mitigar o problema de mensagens coladas ou incompletas, foi implementado um buffer_sobras do tipo binário (b"). O servidor acumula os dados recebidos pelo método `recv(1024)` e busca ativamente pelo delimitador de quebra de linha (binário `\n`). Somente quando uma mensagem completa é isolada, ela é decodificada em UTF-8 e repassada à camada superior.

ControladorFazenda (ClassControlador.py)

Responsabilidade: Atuar como o Dispatcher (roteador) central do jogo. Traduz strings brutas em objetos JSON, valida a estrutura sintática, invoca as regras de alteração do mundo e formata as respostas adequadas.

Funcionalidades e Permissões: Funciona como a ponte lógica ligando a Rede, o Mapa e os Usuários. É o único componente com permissão para capturar exceções sistêmicas geradas por ações simultâneas dos jogadores por meio de um mecanismo interno de proteção de fluxo.

Escolha(s) de Implementação Crítica (Escudo Anti-Crash e log): Para garantir a tolerância a falhas do servidor contra condições de corrida causadas por cliques excessivamente rápidos ou comandos mal formados, o método *processa_mensagem* é inteiramente envelopado em um bloco try/except Exception. Caso ocorra um erro inesperado (como falhas de variáveis locais não associadas ou corrupção de estado), o erro é loggado via `traceback.print_exc()` e uma mensagem de erro estruturada em JSON é retornada ao cliente de forma segura, impedindo a queda imprevista da thread do servidor ou do serviço global. Também é feita um log por terminal para se ter melhor noção do das causas:

```
except Exception as e:
    import traceback
    print("\n" + "="*50)
    print("ESCUDO ANTI-CRASH ATIVADO NO SERVIDOR")
    print(f"0 jogador {id_jogador} clicou rápido demais e gerou um erro interno!")
    print(f"JSON que causou a falha: {mensagem_str}")
    print("--- Detalhes do Erro ---")
    traceback.print_exc() # Isso vai imprimir a exata linha de código que quebrou!
    print("\n" + "="*50 + "\n")

# Devolvemos um erro seguro para o Cliente, para a interface dele não desconectar
resposta_segura = {
    "comando": "RESPOSTA",
    "status": "ERRO",
    "mensagem": "Nao espame clickes, jogue na calma !!!."
}
return json.dumps(resposta_segura) + "\n", None
```

Camada Models

Estado do Jogo/Mapa (Class Mapa.py)

Responsabilidade: Manter a matriz bidimensional representativa do mundo físico do jogo e aplicar as restrições geográficas de cultivo.

Funcionalidades e Permissões: Controla de forma exclusiva as mutações dos blocos (Tiles). É responsável por gerar proceduralmente elementos como oásis (água cercada de areia), rios (verticais ou horizontais) e poças.

```
#aleatorizando mapa
num_oasis = random.randint(3, 6)
direcao = random.choice(["horizontal", "vertical"])
num_nascentes = random.randint(3, 6)
prob_areia = random.uniform(0.3, 0.5) # Probabilidade de cada célula ser areia
num_pocas = random.randint(3, 6)
maxtam_pocas = random.randint(2, 4)

mapa_jogo.gerar_mapa_aleatorio(num_oasis, direcao, num_nascentes,
                               prob_areia, num_pocas, maxtam_pocas)
mapaV.MapaView.exibir_colorido(mapa_jogo.matriz)
```

Regras de Negócio Implementadas:

- Trigo (ID 3): Só pode ser plantado em blocos de Terra (ID 0).
- Arroz (ID 4): Só pode ser plantado exclusivamente em blocos de Água (ID 1).
- Cana-de-Açúcar (ID 5): Pode ser plantada em Terra ou Areia, transformando-se dinamicamente em ID 6 (Cana na Terra) ou ID 7 (Cana na Areia), respectivamente. Essa transformação dinâmica serve pra conseguir voltar para o estado anterior antes de plantar sem alterar a composição original do mapa

GerenciadorUsers e User (ClassUser.py)

Responsabilidade: Gerenciar as credenciais voláteis (Nicks) dos jogadores, seus inventários individuais de colheita e o controle de vagas ativas no servidor.

Funcionalidades e Permissões: Limita o acesso ao servidor a no máximo 4 jogadores simultâneos por meio de um mapeamento estático de slots (1 a 4). Realiza a alocação de nomes temporários (ex: "Jogador1") no momento da conexão, permitindo a posterior alteração por meio de requisições específicas.

Camada Views:

ClassMapViewTerminal.py

Trata-se apenas da visualização por meios de terminal para vermos um pré-visualização do mapa gerado, no terminal do servidor, que será renderizado pelo lado do cliente.

mainServer.py

Responsabilidade: Orquestrar a inicialização das estruturas, gerar o mapa inicial aleatório, instanciar as classes de dados, injetar as funções de gatilho (*callbacks*) cruzadas entre o Controlador e o Servidor TCP, e finalmente disparar o laço principal de escuta da rede.

Componentes (Cliente)

Camada Controller:

ClassControladorCliente.py e ClassOuvinte.py

Responsabilidade: Funciona como o núcleo lógico do cliente, encarregado de intermediar a interação entre a View e o Model de forma bidirecional.

Mapeamento Evento-Ação: O controlador intercepta o acionamento de cliques originados da View através de conexões de sinais do PyQt (*cellClicked.connect*). Ele infere dinamicamente o estado atualizado dos botões (Modo Plantar/Colher e Semente ativa) e aciona o método equivalente na camada Model para envio à rede. No fluxo inverso, escuta as notificações distribuídas pela Thread de escuta do sistema e invoca os métodos de pintura(mudar cor das células da matriz) e logs pertinentes na Janela Principal da View.

Camada Models:

Comunicação Base e Estruturação

TCPCliente (ClassTCPCliente.py):

Representa o invólucro de abstração da API de Sockets de baixo nível em modo síncrono. Sua responsabilidade restringe-se a estabelecer conexões físicas via `socket.connect()`, despachar strings em formato binário encapsuladas pelo método `sendall()`, e acumular os fragmentos brutos recebidos da rede em um acumulador persistente de bytes (`buffer_sobras = b""`).

ClienteFazenda (ClassFazendeiro.py):

Atua como a fachada de requisições do cliente (Service Facade). Ele não possui referência para componentes de tela. Sua responsabilidade é receber chamadas de métodos de alto nível de negócio (ex: `solicita_plantar`) e serializá-las em formato JSON padronizado. Da mesma forma, intercepta strings decodificadas de rede e as traduz em estruturas nativas de dicionários Python (Eventos) inteligíveis para o controlador.

Camada View:

Interface Gráfica Pura (ClassGUI.py)

Responsabilidade: Construir, dispor e atualizar os elementos tridimensionais, botões e campos visuais do usuário na tela através do framework PyQt6.

Isolamento Arquitetural: Em conformidade com o padrão MVC, o arquivo ClassGUI.py é inteiramente agnóstico à rede. Ele não faz importações do módulo *socket* ou de *JSONs* e não sabe da existência de servidores. Classes gráficas como *MatrixCell* (que representam individualmente cada quadrado do mapa) e *GameWindow* expõem sinais de controle e métodos públicos de mutação gráfica (como *atualizar_visual(valor)*) para permitir que agentes externos governam seu comportamento estético.

mainClient.py

Responsabilidade: É uma instância cliente, podendo (por terminais distintos) ser chamado mais de uma vez a fim de orquestrar entre o que escuta do servidor e o que manda, enquanto instância uma interface gráfica que facilita e embeleza o mapa recebido.

Troca de Mensagens

A troca de mensagens baseia-se estritamente no envio de strings formatadas em JSON, finalizadas obrigatoriamente pelo caractere terminador de linha `\n`. O sistema trabalha com dois fluxos fundamentais: Requisição-Resposta Unicast (direcionada exclusivamente ao socket emissor) e Broadcast (espalhada para todas as conexões armazenadas na tabela ativa).

As trocas de mensagens podem ser divididas em três momentos:

1. Primeira interação (Maior Carga útil de Dados): Trata-se do usuário pedindo para se conectar com servidor, caso seja aceito essa conexão o servidor utilizará o nickname que o usuário pediu de início e enviará toda a matriz para que possa ser reconstruída exatamente igual ao seu estado atual no servidor, mas na máquina do cliente.

2. **Conversa Bilateral:** Cliente manda solicitações e recebe, estruturadas por JSONs, para o servidor. O servidor ler e manda respostas, também estruturadas por JSONs, para o cliente. As mensagens são estruturadas de maneira que ambos os lados possam saber do que se trata o “assunto” (usando as flags do JSON) e assim saber como lidar e quais funções, ou eventos chamarem.

3. **Broadcast:** Cada mensagem é tratada respondida de forma individual para os usuários (clientes) que as estão trocando com o servidor. Porém há mensagens que desencadeiam um broadcast, já solicitam alteração do mapa hosteado pelo servidor. Assim tais mensagens são respondidas tanto individualmente quanto também replicadas para todos os outros usuários ativos.

Tabela de Mensagens (Servidor para Cliente)

Comando JSON ("comando")	Estrutura Completa do Payload (JSON)	Tipo de Envio	Gatilho / Quando é enviada?	Objetivo e Impacto no Cliente
RESPOSTA (Boas-Vindas)	{ "comando": "RESPOSTA", "status": "OK", "mensagem": "BEM_VINDO: <Nick>" }	Unicast (Apenas para o emissor)	Enviada após o cliente enviar o comando NICKNAME com sucesso.	Confirma que o nome foi registrado no servidor e libera a transição para a tela principal.

RESPOSTA (Sucesso)	{ "comando": "RESPOSTA", "status": "OK", "mensagem": "Plantio realizado com sucesso!" } ou "Colheita realizada com sucesso!"	Unicast (Apenas para o emissor)	Enviada para o jogador que realizou a ação, logo após a validação dar True no mapa.	Serve como um "recibo" individual da ação bem-sucedida, registrando um log de sucesso com  na tela.
RESPOSTA (Erro de Negócio)	{ "comando": "RESPOSTA", "status": "ERRO", "mensagem": "<Motivo da Falha>" } (Ex: "Solo incompatível", "Longe da agua", "Nao ha essa plantacao...")	Unicast (Apenas para o emissor)	Enviada imediatamente quando a ação de plantar ou colher viola alguma regra da ClassMapa.	Informa o motivo da recusa para o jogador, adicionando um log com  sem alterar os blocos visuais.
RESPOSTA (Erro de Concorrência)	{ "comando": "RESPOSTA", "status": "ERRO", "mensagem": "Nao espame clickes, jogue na calma !!!." }	Unicast (Apenas para o emissor)	Disparada pelo Escudo Anti-Crash caso cliques simultâneos gerem uma exceção imprevista na Thread.	Impede que a interface gráfica trave ou desconecte se houver condições de corrida extremas.
RESPOSTA (Servidor Cheio)	{ "comando": "RESPOSTA", "status": "ERRO", "mensagem": "Servidor cheio" }	Unicast (Apenas para o emissor)	Enviada na hora do aperto de mão técnico (accept) se já houver 4 jogadores ativos.	Interceptada pelo cliente para disparar um pop-up crítico de acesso negado e fechar o app de forma limpa.

RESPOSTA (Desconexão)	{ "comando": "RESPOSTA", "status": "OK", "mensagem": "Desconectando..." }	Unicast (Apenas para o emissor)	Enviada em resposta ao comando voluntário SAIR enviado pelo cliente.	Avisa o cliente que o encerramento do socket foi autorizado pelo backend.
ATUALIZAR_MAPA	{ "comando": "ATUALIZAR_MAPA", "matriz": [[0,0,1...], [0,2,...]] }	Unicast (Apenas para o emissor)	Solicitada pelo cliente através do comando MATRIZ logo após o login do jogador.	Transmite a matriz numérica integral do terreno (20x20) para renderizar o cenário completo na tela.
ATUALIZAR_CELULA	{ "comando": "ATUALIZAR_CELULA" , "x": <int>, "y": <int>, "valor": <int> }	Broadcast (Para todos os conectados)	Desparada de forma assíncrona sempre que qualquer jogador planta ou colhe algo com sucesso.	Força a interface gráfica de todos os fazendeiros online a repintar a célula (x, y) com o novo ID em tempo real.
NOVO_JOGADOR	{ "comando": "NOVO_JOGADOR", "nome": "<Nome do Jogador>" }	Broadcast (Para todos os conectados)	Disparada no exato milissegundo em que um novo jogador define seu Nickname com sucesso.	Faz com que o chat/log de todos os outros clientes exiba uma notificação avisando que um novo fazendeiro entrou.

JOGADOR_SAIU	{ "comando": "JOGADOR_SAIU", "nome": "<Nome do Jogador>" }	Broadcast (Para todos os conectados)	Disparada quando um jogador clica para fechar o jogo ou o servidor detecta sua saída voluntária.	Alerta os demais fazendeiros de que aquele jogador se desconectou e liberou uma vaga no servidor.
--------------	---	--	--	--

```

#-----
def gera_boas_vindas(self, id_jogador): ...

def resposta_nickname(self, id_jogador, dados): ...

#-----
def resposta_plantar(self, id_jogador, dados): ...

#-----
def resposta_colher(self, id_jogador, dados): ...

#-----
def resposta_matriz(self): ...

#-----
def resposta_sair(self, id_jogador): ...

#-----
def resposta_erro(self): ...

#-----
def processa_mensagem(self, id_jogador, mensagem_str): ...

```

Tabela de Mensagens (Cliente para Servidor)

Comando (Origem)	Payload do JSON Enviado / Tratado	Tipo de Fluxo de Rede	Impacto no Estado e Resposta
NICKNAME (Cliente)	{"comando": "NICKNAME", "nome": "Gabriel"}	Unicast (Confirmação) + Broadcast (Notificação)	Altera o nome do jogador no slot correspondente. O servidor responde com um Unicast de boas-vindas e dispara um Broadcast informando aos demais: {"comando": "NOVO_JOGADOR", "nome": "Gabriel"}.
PLANTAR (Cliente)	{"comando": "PLANTAR", "semente": 3, "x": 4, "y": 13}	Unicast (Laudo) + Broadcast (Se Válido)	O mapa valida as coordenadas e o tipo de semente. Se válido, altera a matriz do servidor e emite um Broadcast: {"comando": "ATUALIZAR_CELULA", "x": 4, "y": 13, "valor": 3}. Em caso de falha (ex: spam na mesma célula), envia um Unicast de erro ao emissor.
COLHER (Cliente)	{"comando": "COLHER", "cultura": 3, "x": 4, "y": 13}	Unicast (Laudo) + Broadcast (Se Válido)	Verifica se a cultura alvo coincide com a célula modificada. Se bem-sucedido, restaura o terreno original da célula (Terra/Água/Areia) e emite um Broadcast de atualização de célula para sincronizar todas as telas.
MATRIZ (Cliente)	{"comando": "MATRIZ"}	Unicast Estrito	Utilizado para sincronização completa quando um cliente inicia. O servidor devolve a matriz 2D bruta em formato JSON: {"comando": "ATUALIZAR_MAPA", "matriz": [[...]]}.
SAIR (Cliente)	{"comando": "SAIR"}	Unicast (Confirmação) + Broadcast (Desconexão)	Sinaliza a saída voluntária do cliente. O servidor libera a vaga no gerenciador de usuários, fecha o socket e envia aos demais: {"comando": "JOGADOR_SAIU", "nome": "Gabriel"}.

```

def solicita_nickname(self, nome): ...

def solicita_plantar(self, semente, x, y): ...

def solicita_colher(self, cultura, x, y): ...

def solicita_matriz(self): ...

def solicita_sair(self): ...

def escutar_servidor(self):|...
```

Uso das Threads

Para suportar múltiplos usuários jogando e interagindo simultaneamente no mesmo espaço “geográfico”, o uso de múltiplas linhas de execução (*threads*) tornou-se mandatório no Servidor

No lado do cliente, o uso da threads foi essencial para implementar uma maneira de receber mensagens do servidor sem que os usuários ficassem travados e impossibilitados de efetivamente jogar.

Threads no Servidor:

Thread Principal (Mestre): Permanece travada no laço infinito de escuta do método *server_socket.accept()* dentro da classe *ServidorTCP*. Sua única função é interceptar novas tentativas de conexões de rede.

Threads de Clientes (Trabalhadoras / Workers): Assim que o método *accept()* detecta um novo aperto de mão (handshake) e válida que há vagas disponíveis no gerenciador de usuários (*slots < 4*), o servidor instância e dispara uma nova thread nativa utilizando a biblioteca *threading.Thread*, apontando o alvo (*target*) para o método interno *trata_cliente* e passando o *socket* específico como argumento.

Mecanismos de Sincronização e Concorrência; GIL (Global Interpreter Lock): Como as operações internas nas matrizes e dicionários em nível de CPU ocorrem em blocos de bytecode protegidos pelo GIL, o servidor consegue receber múltiplos comandos simultâneos (ex: requisições consecutivas de plantio de sementes) sem corromper as estruturas de dados fundamentais do Python. O roteamento imediato dos laudos de validação de sucesso ou falha (*{"validar": True/False}*) impede que dois blocos ocupem o mesmo espaço de forma inválida.

Threads no Cliente:

Nenhuma operação bloqueante ou demorada pode ser executada na Thread Principal, sob o risco de congelar a renderização da tela e tornar o programa inoperante para o sistema operacional.

A Thread Oculta de Escuta (OuvinteThread)

Para contornar o travamento gerado pela chamada ao método de rede `socket.recv()` (que aguarda indefinidamente por pacotes originados do servidor), foi concebida a classe *OuvinteThread*. Esta classe estende a linha de execução nativa do framework Qt através de `QThread`.

- **Funcionamento:** Ao iniciar o jogo, a *OuvinteThread* é disparada em paralelo à tela. Ela executa um laço contínuo de escuta em segundo plano no método `run()`, chamando a função bloqueante da rede: `self.fazendeiro.escutar_servidor()`.
- **Barreira de Isolamento:** Esta Thread "fantasma" nunca altera os botões ou as cores da tela diretamente. Modificar elementos visuais a partir de uma thread secundária viola os princípios de concorrência do PyQt6 e gera falhas de segmentação (*Segmentation Faults*).

Mecanismo de Sincronização Segura via Sinais (pyqtSignal)

Para transmitir os dados recebidos da rede em segundo plano para a Thread da Interface Gráfica de forma segura e sincronizada, utilizou-se o barramento de eventos nativo do Qt baseado em Sinais e Slots.

- A classe *OuvinteThread* declara um ponto de transmissão tipado:
`signal_evento = pyqtSignal(dict)`.

```
def run(self):
    while self.running:
        evento = self.fazendeiro.escutar_servidor()
        self.signal_evento.emit(evento)

        if evento["tipo"] == "DESCONECTADO":
            self.running = False
            break
```

- Ao receber um pacote íntegro do servidor, a thread converte a mensagem e aciona a emissão de sinal: `self.signal_evento.emit(evento)`.

- Na inicialização do sistema, o controlador vinculou o sinal ao método de processamento: `thread_escuta.sinal_evento.connect(self.processar_evento_rede)`.
- **Sincronização implícita:** O framework PyQt6 intercepta a emissão do sinal através de uma fila interna de mensagens (*Event Queue*). Ele transfere o dicionário de dados da linha de execução secundária e invoca o método do controlador com segurança dentro do contexto da Thread Principal. Isso garante uma sincronização livre de conflitos (Thread-Safe) sem a necessidade de travas manuais complexas (*Mutexes/Locks*) na camada gráfica do cliente.

```

it > views > ClassGUI.py > GameWindow > processar_mensagem_servidor
def processar_mensagem_servidor(self, evento: dict):
    """Recebe o dicionário da Thread de Rede e atualiza a interface."""
    tipo = evento.get("tipo")

    if tipo == "IGNORAR":
        return # Não faz nada, apenas volta a ouvir

    elif tipo == "MAPA_COMPLETO": ...

    elif tipo == "CELULA": ...

    elif tipo == "RESPOSTA_SISTEMA":
        msg = evento.get("mensagem")
        status = evento.get("status")

        if status == "ERRO":
            if msg == "Servidor cheio":
                QMessageBox.critical(
                    self,
                    "Acesso Negado",
                    "A fazenda já atingiu o limite máximo de agricultores (4/4).\n\nTente cone
                )
                self.close() # Fecha a janela do jogo e encerra o programa
            else:
                self.log_message("Sistema", f" {msg}")
        else:
            self.server_message(f" {msg}")

    elif tipo == "NOVO_JOGADOR": ...

    elif tipo == "JOGADOR_SAIU": ...

```

Dificuldades:

1. Achar um meio para padronizar as trocas de mensagens distintas, de grau de complexidade variante e com finalidades diversas sem sacrificar muito aumentando o tamanho e a latência.
 - a. Achar flags e maneiras de fazê-las abrangentes e não prolixas.
2. Tratar erros e prever possíveis cases relacionados a conexão de sockets.
 - a. A condição de corrida, especialmente quando um usuário dava muitos clicks, trouxe muita dor de cabeça. Mas foi tratado com uma percepção dessa ação (SPAM) e o servidor ignorando
 - b. A dificuldade acima possibilitou a ideia de uma tratativa geral de falhas , onde travadas (trashs) e ações fora do escopo eram tratadas como erro, sem precisão de cortar a thread com o usuário.
 - c. buffer de mensagens cheio ou mensagens chegando muito rápido (2 em um só receb() por exemplo). Resolução foi um buffer temporario e binario, ja que se o buffer cortasse o \n poderia descartar mensagens que estiverssem no meio: (EX M1\nM2\n)
3. Desenvolver um front-end, e ainda usar threads juntamente com o front-end. A dificuldade veio de nunca ter feito um front-end, e em descobrir que eles tem suas “regras” para lidar com threads.

Extras

No projeto inicial havia um requisito de fazer com o cliente de tempo em tempo atualizasse o mapa (matriz) por broadcast do servidor, ou podia solicitar fora do time essa atualização. Devido ao melhor entendimento do funcionamento dos soquetes e da rede, foi possível excluir esse requisito e deixando uma atualização contínua para todos os jogadores, deixando o mapa sempre em seus estados mais atuais.

O melhor entendimento também possibilitou uma melhor tratativa das falhas, já que antes o idealizado era tratar poucos casos, e na prática conseguimos não só tratar, mas também implementar redundâncias e transparência de forma que o usuário até possa ver algumas logs, mas não seja afetado com isso

Conclusão

Os resultados obtidos durante a execução e os testes de estresse do sistema foram considerados altamente válidos e satisfatórios. A implementação de um protocolo de comunicação semântico baseado em JSON, aliado ao mecanismo de buffer persistente de bytes, superou de forma definitiva os problemas físicos de fragmentação e aglutinação de pacotes inerentes à natureza de fluxo contínuo (stream) do TCP.

Do lado do cliente, o isolamento lógico das rotinas blocantes de rede em threads secundárias (QThread) e a comunicação segura com a interface gráfica através do barramento de eventos (pyqtSignal) garantiram uma experiência de usuário fluida e livre de travamentos (freezes), respeitando as diretrizes de concorrência do framework PyQt6.

Adicionalmente, as estratégias de tolerância a falhas implementadas — em especial o "Escudo Anti-Crash" global do servidor e a validação descentralizada de estado — provaram ser extremamente eficazes. O sistema manteve a consistência global e a estabilidade da topologia de rede mesmo sob condições de altíssima concorrência e anomalias de uso (como o spam de cliques e envio excessivo de requisições por parte dos clientes).

Em suma, o projeto atendeu a todos os requisitos funcionais propostos para a simulação agrícola multiplayer (sincronização de matrizes, gerenciamento de lotação, regras de plantio e colheita), mas também ampliou consolidando com êxito os fundamentos teóricos e práticos de Sistemas Distribuídos, resultando em uma aplicação robusta, resiliente e pronta para ambientes containerizados.

Github:

None

[git@github.com:deyrik/Garden-in-Overworld.git](https://github.com/deyrik/Garden-in-Overworld.git)